GM / Hummer H3 BCM Mileage Algorithm Reference

Reverse Engineering & Technical Diagnostics Report

July 17, 2026

Abstract

This document outlines the engineering specification, mathematical logic, and physical memory structures governing the odometer storage system within General Motors Body Control Modules (BCM) for the Hummer H3 platform (model years 2005–2009). It details how the mileage is calculated, converted, and stored across the AT25040 serial EEPROM memory space.

1 Introduction

In the GM/Hummer H3 architecture, the instrument cluster's odometer display acts as a passive client to the central Body Control Module (BCM). The cluster retrieves, verifies, and synchronizes its mileage data from the BCM.

When the instrument cluster detects a cleared odometer memory state (such as a zeroed 93C56 EEPROM), it initiates an immediate query to the BCM. If the BCM contains valid, structured mileage data, the cluster automatically updates its local display, completing a self-repair and alignment cycle.

2 The Core Odometer Algorithm

The BCM does not store mileage as a raw integer or in metric kilometers. Instead, it tracks cumulative travel using high-resolution **Vehicle Speed Sensor (VSS) pulses**.

In this specific GM architecture, the VSS scalar constant is defined as exactly:

$$\text{VSS Constant} = 4,000 \text{ pulses per mile} \tag{1}$$

2.1 Encoding Formula

To encode a target mileage value into the 32-bit big-endian hexadecimal representation required by the BCM EEPROM:

$$\text{Pulses}_{\text{decimal}} = \text{Miles} \times 4000 \tag{2}$$

Convert the resulting decimal pulses to a 32-bit hexadecimal value:

$$\text{Pulses}_{\text{decimal}} \longrightarrow \text{Hex}_{\text{32-bit Big-Endian}} \tag{3}$$

2.2 Decoding Formula

To reconstruct the physical mileage from a raw EEPROM hex dump:

$$\text{Hex}_{\text{32-bit}} \longrightarrow \text{Pulses}_{\text{decimal}} \tag{4}$$

$$\text{Miles} = \frac{\text{Pulses}_{\text{decimal}}}{4000} \tag{5}$$

3 Empirical Validation & Proofs

The mathematical validity of this algorithm is confirmed by two distinct empirical datasets extracted from working modules.

3.1 Proof 1: The 180,705 Mile Case

Starting with the target mileage shown in diagnostic tools:

- **Target Miles:** $180,705$
- **Calculate Pulses:**
$$180,705 \times 4000 = 722,820,000 \text{ pulses}$$
- **Decimal to Hexadecimal Conversion:**
$$722,820,000_{10} = \text{0x2B155BA0}$$
- **Big-Endian Byte Structure:**
$$\text{Byte 0: } \texttt{2B}, \quad \text{Byte 1: } \texttt{15}, \quad \text{Byte 2: } \texttt{5B}, \quad \text{Byte 3: } \texttt{A0}$$

This perfectly matches the highlighted segment observed in the physical BCM dump: `2B 15 5B A0`.

3.2 Proof 2: The 100,000 Mile Default Standard

Examining the default initialization patterns from historical calibration libraries:

- **Raw Hex String:** `17 D7 84 00`
- **Hexadecimal to Decimal Conversion:**
$$\text{0x17D78400} = 400,000,000 \text{ pulses}$$
- **Divide by VSS Scalar:**
$$\frac{400,000,000}{4000} = 100,000 \text{ miles}$$

This validates that the default firmware structures represent a baseline value of exactly $100,000$ miles.

4 BCM Memory Map & Storage Structure

The target BCM memory device is an STMicroelectronics or Atmel **AT25040** serial EEPROM (organized as 512×8 bits).

To prevent data corruption from localized electrical fluctuations or memory wear, GM utilizes **Triple Modular Redundancy (TMR)**. The 4-byte encoded mileage payload must be written sequentially three times at memory block address `0x0080`.

Table 1: BCM Odometer Register Map (Address 0x0080)

Byte Offset	Data Structure	Description
0x80 – 0x83	2B 15 5B A0	Copy 1 (Primary Value)
0x84 – 0x87	2B 15 5B A0	Copy 2 (Redundant Verification)
0x88 – 0x8B	2B 15 5B A0	Copy 3 (Redundant Verification)
0x8C – 0x8F	XX XX XX XX	System Checksum & Status Registers

5 Diagnostic Procedure

When deploying a replacement instrument cluster or BCM, perform the following steps to ensure perfect system alignment:

1. Zero out the cluster's 93C56 EEPROM (specifically lines 0x00D0 and 0x00F0) by replacing all active bytes with 00 00 00 00.
2. Extract the BCM AT25040 EEPROM dump.
3. Calculate the target odometer value using the formula Miles $\times$ 4000.
4. Overwrite the 12 bytes starting at BCM address 0x0080 with the triple-redundant hex stream.
5. Flash the modified dump back to the BCM.
6. Power up the vehicle. The cluster will query the BCM, discover the clean register, and securely copy the new value.